

PeakCheck — Technical Supplement

Method, algorithms and annotated source code

Lukas Knauer

Volker Schünemann

RPTU Kaiserslautern-Landau · AG Schünemann

Version 1.0.0 · MIT license · 2026

This supplement derives every computational step of PeakCheck and shows the corresponding excerpts of the `peakcheck` package — closely following the source, with brief inline comments added for orientation and longer passages abbreviated by `...` — each followed by a short explanation, so the code can be followed directly. Every method is backed by a literature reference (Section 19); the same references appear as NumPy-style docstrings in the source. Code excerpts carry a header naming the exact submodule and function, e.g. `peakcheck/profiles.py -- voigt()`, so each formula can be located in the source at a glance.

Contents

1	Overview and code map	2
2	Configuration (Config / TOML)	3
3	Reading and noise estimation	4
3.1	Axis transform: affine and reciprocal	4
4	Line profiles — the true Voigt	5
4.1	Why the Faddeeva function yields the Voigt	6
5	Background subtraction	7
6	Peak search	8
7	NNLS amplitude fit	9
7.1	Fit window and model assembly	11
7.2	The active-set method (NNLS)	11
8	Goodness of fit and standard errors	12
9	Presence check in the components	14
9.1	The three stages in code	14
10	Graphical interface	15
11	Outputs and reproducibility	16
11.1	Presence matrix with colour coding	16
11.2	Plain ASCII CSV with a parameter header	17
11.3	What the parameter block contains	18

12	Worked example	18
13	Numerical validation and exactness	19
13.1	Monte-Carlo recovery and error calibration	20
14	Assumptions, limitations and parameter selection	20
15	Relation to nonlinear least-squares fitting	21
16	Error budget: a worked covariance example	23
17	Computational complexity and performance	24
18	Parameter reference	25
19	References	25
20	Notation	27

1 Overview and code map

PeakCheck answers a single question: *which peaks occur in the reference data set, and which of them are present in the component data sets?* The flow is linear and bundled in `run_analysis()`; the graphical interface (`PeakCheckGUI`) and the batch mode (`run_headless()`) call the same computational core.

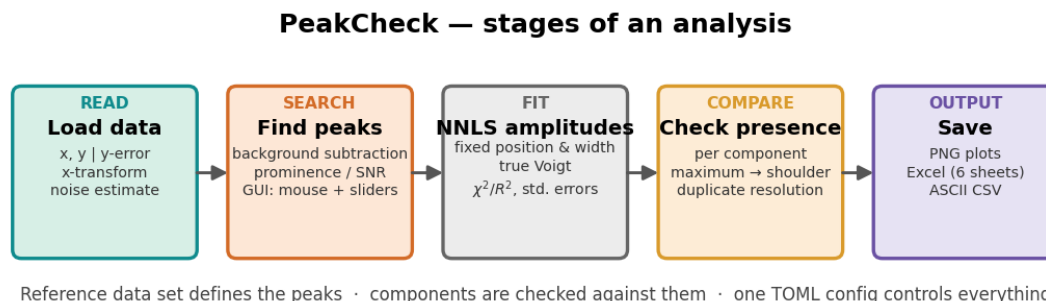


Figure 1. The five stages of an analysis. The GUI and the headless run share the same computational core.

The map below assigns the responsible functions to each stage; the module table that follows names the submodule holding each function.

The code is structured as a Python package — each stage lives in its own submodule, all sharing the same `Config` and feeding the same `run_analysis()` core:

The public surface is re-exported from `peakcheck/__init__.py`, so from `peakcheck` import `voigt`, `fit_amplitudes`, `run_analysis` keeps working. The `peakcheck.py` file at the project root is a thin launcher that forwards to the package.

Stage	Function(s)	Task
Read	<code>load_xy</code> , <code>estimate_noise</code>	read the file robustly, transform x , estimate the noise
Profiles	<code>voigt</code> , <code>gaussian</code> , <code>lorentzian</code> , <code>profile_shape</code>	area-normalised line shape
Background	<code>subtract_background</code>	remove the baseline as a rolling minimum
Search	<code>search_peaks</code> , <code>GUI_on_click</code>	find peaks automatically / place them by mouse
Fit	<code>fit_amplitudes</code> , <code>make_fit_mask</code> , <code>compute_model</code>	NNLS amplitudes at fixed positions/widths
Quality	<code>fit_statistics</code>	χ^2/R^2 and standard errors
Compare	<code>check_peaks_in_component</code>	presence per component
Output	<code>save_excel</code> , <code>save_csv</code> , <code>gather_parameters</code>	plots, Excel, ASCII CSV, metadata

Module	Contents
<code>peakcheck.config</code>	<code>Config</code> , <code>X_CONVERSIONS</code> , TOML I/O, axis transforms
<code>peakcheck.profiles</code>	true Voigt, Gaussian, Lorentzian, <code>profile_shape</code>
<code>peakcheck.io</code>	<code>load_xy</code> , <code>estimate_noise</code>
<code>peakcheck.background</code>	<code>subtract_background</code> , <code>x_range_mask</code>
<code>peakcheck.fit</code>	<code>search_peaks</code> , <code>make_fit_mask</code> , <code>fit_amplitudes</code> , <code>fit_statistics</code>
<code>peakcheck.presence</code>	<code>check_peaks_in_component</code>
<code>peakcheck.plots</code>	<code>plot_fit_result</code> , <code>plot_component</code> , <code>_outpath</code>
<code>peakcheck.output</code>	<code>gather_parameters</code> , <code>save_excel</code> , <code>save_csv</code>
<code>peakcheck.pipeline</code>	<code>run_analysis</code> , <code>run_headless</code> , <code>find_component_files</code>
<code>peakcheck.gui</code>	PeakCheckGUI (Tkinter)
<code>peakcheck.cli</code>	CLI: <code>main</code> , <code>build_arg_parser</code>

2 Configuration (Config / TOML)

All parameters live in a `@dataclass Config` with defaults and helper methods. The widths `wg` (Gaussian FWHM) and `wl` (Lorentzian FWHM) are turned into the profile parameters by `sigma()` and `gamma()`:

$$\sigma = \frac{f_G}{2\sqrt{2\ln 2}} \quad (\text{sigma}()), \quad \gamma = \frac{f_L}{2} \quad (\text{gamma}).$$

`peakcheck/config.py -- Config.effective_fwhm()` (total FWHM of the Voigt profile)

```
def effective_fwhm(self) -> float:
    p = str(self.profile).lower()
    if p in ("gauss", "gaussian"):
        return self.wg
    if p in ("lorentz", "lorentzian"):
        return self.wl
    return 0.5346 * self.wl + np.sqrt(0.2166 * self.wl ** 2 + self.wg ** 2)
```

The Voigt FWHM has no closed form; the empirical approximation of Olivero & Longbothum [5] (accurate to about 0.02 %) is used. `load_config()` reads the TOML file section by section and flattens the tables into the flat `Config`; `write_template()` writes a commented template. Since TOML has no `null`, empty strings for `x_min`/`x_max` are turned back into `None` by `_coerce_nullable()` (Section 18).

Because the GUI, the headless run and the TOML all build on the same `Config`, a given parameter set always yields the same result, independent of the entry path — the basis of reproducibility

(Section 11).

3 Reading and noise estimation

`load_xy()` reads text files with two or three columns. A fast path (`numpy.loadtxt`) is tried first; on failure a tolerant line-by-line parser takes over, coping with header lines, comment characters (`## ! \%`), comma decimal separators and extra columns. The file extension does not matter (`.dat`, `.txt`, `.csv`, `.xy`, `.asc`, `.tsv`, `.prn`, `.nis`, ...); values may be separated by whitespace, tabs, commas or semicolons. A German-style decimal comma is distinguished from a separator comma by anchoring on the whitespace-token count: if every whitespace-separated token is a number once its comma is read as a decimal point, the commas are decimal points; otherwise they are field separators (so `10,5 100,2` reads as two values while `1,2,3` reads as three). Non-finite rows are dropped, the data are sorted by ascending x , and the transform is applied (affine or reciprocal, Section 3.1). Files ending in `.fio` (DESY/PETRA-III format) take a separate path that parses the `\%d` data block and its `Col` declarations and selects the x/y columns via `fio_x_column/fio_y_column`; the result then follows the same transform, sorting and noise handling. The third column is optional (`error_column`); if it is absent, `estimate_noise()` estimates a constant noise robustly from the first differences:

```
peakcheck/io.py -- estimate_noise() (robust noise estimate without an error column)
```

```
def estimate_noise(y):
    d = np.diff(np.asarray(y, dtype=float))
    if len(d) == 0:
        return 1.0
    mad = np.median(np.abs(d - np.median(d)))
    sigma = 1.4826 * mad / np.sqrt(2.0)
    if not np.isfinite(sigma) or sigma <= 0:
        sigma = np.std(y) if np.std(y) > 0 else 1.0
    return float(sigma)
```

$$\sigma = 1.4826 \cdot \frac{\text{MAD}(\Delta y)}{\sqrt{2}}, \quad \Delta y_i = y_{i+1} - y_i.$$

- `np.diff(y)` forms Δy ; differencing suppresses smooth signal so that only the high-frequency noise remains.
- The factor 1.4826 turns the MAD into a consistent estimator of the standard deviation of normally distributed noise [7].
- $1/\sqrt{2}$ corrects the variance doubling from differencing ($\text{Var}(\Delta y) = 2\sigma^2$).
- The fallback to `std(y)/1.0` guarantees a strictly positive σ so the weighted fit never divides by zero.

The return flag `had_errors` records whether real errors were present — it drives the error interpretation in Section 8.

3.1 Axis transform: affine and reciprocal

The x -axis is converted on load. `apply_x_transform()` supports two types (with $a = x_{\text{scale}}$, $b = x_{\text{offset}}$):

```
peakcheck/config.py -- apply_x_transform() (affine and reciprocal conversion)
```

```
def apply_x_transform(x, cfg):
    if str(cfg.x_transform).lower() == "reciprocal":
        x = np.asarray(x, dtype=float)
        safe = np.where(x == 0.0, np.nan, x)
```

```

    return cfg.x_scale / safe + cfg.x_offset
    return x * cfg.x_scale + cfg.x_offset

```

$$\text{affine: } x' = ax + b, \quad \text{reciprocal: } x' = \frac{a}{x} + b.$$

- The affine form covers all linear unit changes, since energy, frequency and wavenumber are proportional. NIS example: `x_scale = 8.065544` converts meV to cm^{-1} .
- The reciprocal form covers wavelength $\leftrightarrow$ wavenumber/energy/frequency, which are *not* linear, e.g. $\tilde{\nu} [\text{cm}^{-1}] = 10^7 / \lambda [\text{nm}]$. The line `np.where(x==0, np.nan, x)` guards against division by zero.
- A reciprocal map reverses the order, so `load_xy()` re-sorts by ascending x' afterwards.

Named presets live in the constant `X_CONVERSIONS` and are applied through `apply_conversion()`, which sets factor, offset, transform type, axis label and unit in one step:

`peakcheck/config.py -- apply_conversion()` (apply a named preset)

```

def apply_conversion(cfg, label) -> bool:
    if label not in X_CONVERSIONS:
        return False
    scale, offset, transform, xlabel, xunit = X_CONVERSIONS[label]
    cfg.x_scale, cfg.x_offset = scale, offset
    cfg.x_transform = transform
    cfg.x_label, cfg.x_unit = xlabel, xunit
    return True

```

The GUI shows only the affine presets; the reciprocal ones are reachable through the TOML key `x_conversion` (e.g. "nm -> eV (reciprocal)"). Because every other quantity (`x_min`, `x_max`, `wg`, `wl`, `tolerance`) is applied *after* the transform, they always refer to the target unit.

4 Line profiles — the true Voigt

Each peak is an area-normalised profile (integral 1); this makes the fitted amplitude the integrated intensity directly and comparable across profiles.

The Voigt profile is the convolution of a Gaussian and a Lorentzian and is computed *exactly* through the Faddeeva function $w(z)$ — not a pseudo-Voigt:

`peakcheck/profiles.py -- voigt()` (exact Voigt via the Faddeeva function)

```

def voigt(x, center, sigma, gamma):
    if sigma <= 0.0:
        return (gamma / np.pi) / ((x - center) ** 2 + gamma ** 2)
    z = ((x - center) + 1j * gamma) / (sigma * np.sqrt(2.0))
    return np.real(wofz(z)) / (sigma * np.sqrt(2.0 * np.pi))

```

$$V(x) = \frac{\text{Re}[w(z)]}{\sigma\sqrt{2\pi}}, \quad z = \frac{(x - x_0) + i\gamma}{\sigma\sqrt{2}}, \quad w(z) = e^{-z^2} \text{erfc}(-iz).$$

- The special case `if sigma <= 0` returns the exact pure-Lorentzian limit ($\sigma \rightarrow 0$); the pure-Gaussian limit ($\gamma \rightarrow 0$) is already contained in $w(z)$.
- `wofz` is the Faddeeva function from `scipy.special` [12]; for the definition see Abramowitz & Stegun [1], Sec. 7.1.
- Dividing by $\sigma\sqrt{2\pi}$ normalises the area to 1.

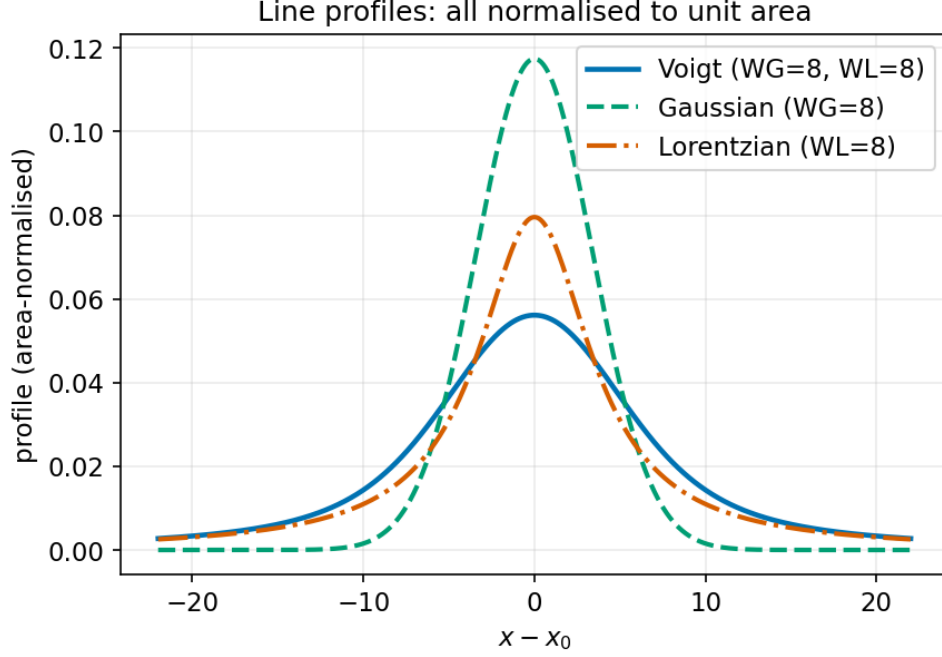


Figure 2. The three selectable profiles, each normalised to unit area.

The pure shapes are dispatched by `profile_shape()` and are equally area-normalised, so amplitudes stay comparable when the profile is switched:

$$G(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left[-\frac{(x-x_0)^2}{2\sigma^2}\right], \quad L(x) = \frac{\gamma/\pi}{(x-x_0)^2 + \gamma^2}.$$

peakcheck/profiles.py -- gaussian(), lorentzian(), profile_shape() (pure shapes + dispatch)

```
def gaussian(x, center, sigma):
    return np.exp(-0.5 * ((x - center) / sigma) ** 2) / (sigma * np.sqrt(2.0 * np.pi))

def lorentzian(x, center, gamma):
    return (gamma / np.pi) / ((x - center) ** 2 + gamma ** 2)

def profile_shape(x, center, sigma, gamma, profile):
    p = str(profile).lower()
    if p in ("gauss", "gaussian"):
        return gaussian(x, center, sigma)
    if p in ("lorentz", "lorentzian"):
        return lorentzian(x, center, gamma)
    return voigt(x, center, sigma, gamma)
```

A pseudo-Voigt $\eta L + (1 - \eta)G$ — used by many programs — gives a visibly different curve:

4.1 Why the Faddeeva function yields the Voigt

The Voigt profile is, by definition, the convolution of a Gaussian line shape (arising from inhomogeneous/instrumental broadening) with a Lorentzian (the homogeneous, lifetime-broadened line),

$$V(x) = (G * L)(x) = \int_{-\infty}^{\infty} G(x') L(x - x_0 - x') dx',$$

with G of standard deviation σ and L of half width at half maximum γ , both centred and area-normalised. This integral has no elementary closed form, but it is exactly the real part of the

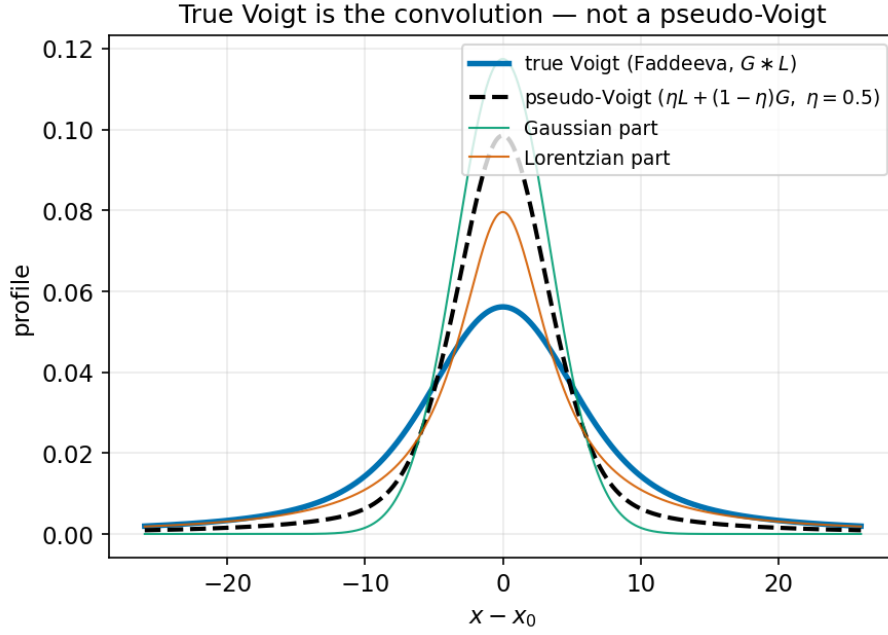


Figure 3. True Voigt (blue) versus pseudo-Voigt (red). PeakCheck uses the true Voigt exclusively.

Faddeeva function $w(z) = e^{-z^2} \operatorname{erfc}(-iz)$ — the scaled complex complementary error function. Writing

$$z = \frac{(x - x_0) + i\gamma}{\sigma\sqrt{2}}, \quad V(x) = \frac{\operatorname{Re}[w(z)]}{\sigma\sqrt{2\pi}},$$

collapses the convolution into a single special-function evaluation. This is exact — no series truncation and no pseudo-Voigt approximation — and numerically stable, because `scipy.special.wofz` evaluates $w(z)$ directly to machine precision (quantified in Section 13). The two limiting cases follow immediately from the same expression: as $\gamma \rightarrow 0$, z becomes real and $\operatorname{Re}[w(z)] = e^{-z^2}$, recovering the pure Gaussian; as $\sigma \rightarrow 0$ (handled by the explicit branch in `voigt()`) the pure Lorentzian remains. The widths of the two components,

$$f_G = 2\sqrt{2\ln 2}\sigma, \quad f_L = 2\gamma,$$

do not simply add; the total Voigt FWHM is given to about 0.02 % by the empirical approximation of Olivero & Longbothum [5], which `effective_fwhm()` uses (Section 2):

$$f_V \approx 0.5346 f_L + \sqrt{0.2166 f_L^2 + f_G^2}.$$

The equivalence is also evident numerically: a direct convolution of a sampled Gaussian and Lorentzian reproduces the Faddeeva result to rounding (Figure 4).

5 Background subtraction

Narrow peaks often sit on a broad pedestal. `subtract_background()` estimates it as a rolling minimum and subtracts it:

peakcheck/background.py -- `subtract_background()` (baseline as a rolling minimum)

```
def subtract_background(y, window):
    if window <= 0:
        return y.copy()
    bg = minimum_filter1d(y, size=window, mode="reflect")
    return np.maximum(y - bg, 0.0)
```

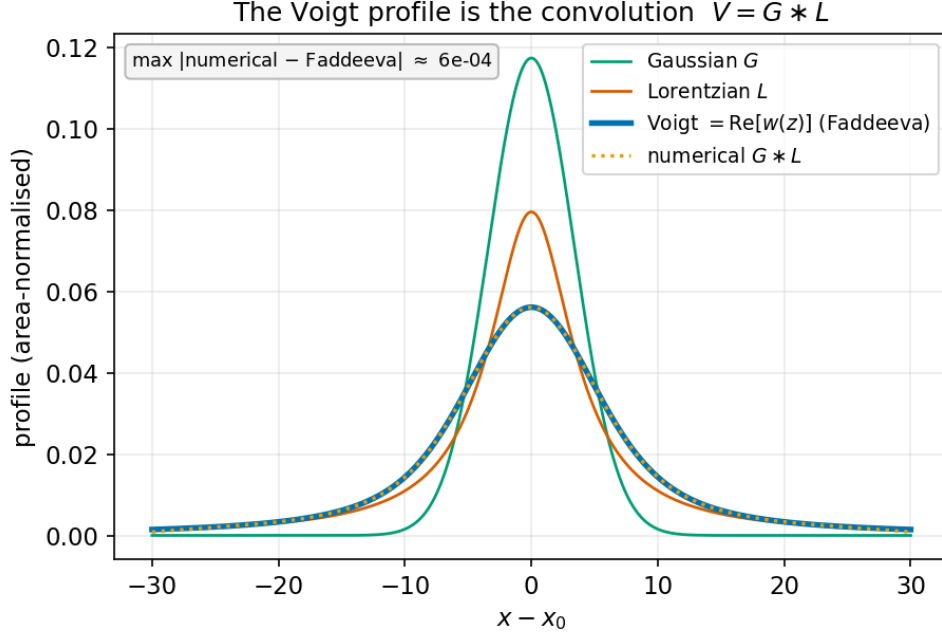


Figure 4. The Voigt profile (blue) *is* the convolution of its Gaussian and Lorentzian components; a direct numerical convolution (dotted) coincides with the Faddeeva evaluation to the displayed tolerance — it is the true Voigt, not a pseudo-Voigt.

$$y_{\text{corr}}(i) = \max\left(y(i) - \min_{|j-i| \leq w/2} y(j), 0\right).$$

- `minimum_filter1d` [12] computes the rolling minimum with window `window`; `mode="reflect"` handles the edges without introducing spurious minima.
- `np.maximum(..., 0.0)` prevents negative values after subtraction.
- `window <= 0` disables the correction (returns a copy).

The fit, the goodness-of-fit statistics, the component presence check and the fit plot all operate on the same configured baseline, so the choice of baseline never desynchronises the quantitative steps; the automatic search and the GUI live preview use the rolling minimum as a candidate-detection aid.

Alternative baselines. The rolling minimum is the default and is robust for narrow peaks on a broad pedestal. Two smooth alternatives are selectable through `baseline_method`: an iterative low-order polynomial ("`polynomial`", ModPoly style) and asymmetric least squares ("`als`", Eilers), the latter governed by a smoothness λ and an asymmetry weight p . Both estimate a smooth baseline that follows broad structure while ignoring the peaks; the default reproduces all earlier results exactly. Figure 6 compares the three on one curved background.

6 Peak search

The automatic search works on the corrected data and filters by signal-to-noise ratio:

`peakcheck/fit.py -- search_peaks()` (automatic peak search with an SNR filter)

```
def search_peaks(y, yerr, x, mask, prominence, distance, min_snr, bg_window):
    y_sub = y[mask]
    yerr_sub = yerr[mask]
    y_corr = subtract_background(y_sub, bg_window)
```

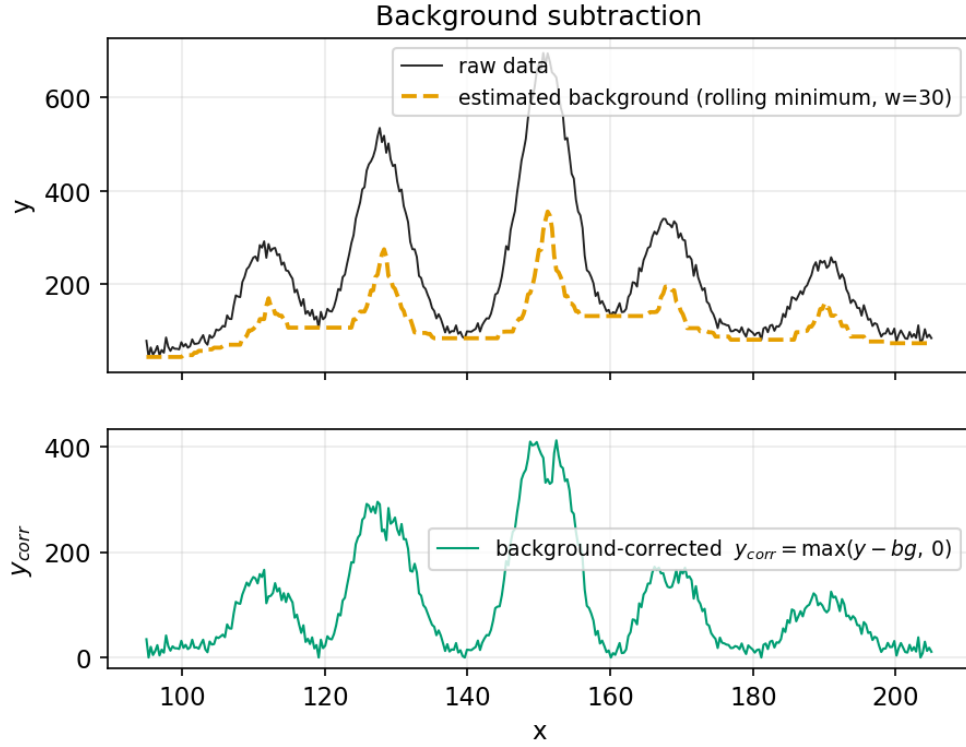


Figure 5. Top: raw data and the background estimated as a rolling minimum. Bottom: the corrected signal.

```

local_idx, _ = find_peaks(y_corr, prominence=prominence,
                          distance=max(1, int(distance)))
if min_snr > 0 and len(local_idx) > 0:
    snr = y_sub[local_idx] / yerr_sub[local_idx]
    local_idx = local_idx[snr >= min_snr]

return np.where(mask)[0][local_idx]

```

- `subtract_background` first removes the baseline so that a sloping background cannot hide weak peaks.
- `find_peaks` [12] returns local maxima, filtered by topographic **prominence** and a minimum **distance**.
- The SNR filter $y_i/\sigma_i \geq \text{min_snr}$ is evaluated on the *raw* data so the threshold reflects the actual measured signal.
- The return value maps the masked-region indices back to the full data set.

Thus a point counts as a peak (*maximum*) only if it is a local maximum in both the corrected and the raw data, stands out, and clears the SNR threshold. A valley (lower than both neighbours) or a spurious peak merely lifted by the subtraction is rejected.

7 NNLS amplitude fit

With fixed positions and widths the fit is linear in the amplitudes $A_k \geq 0$ and is solved as a weighted non-negative least-squares problem (illustrated in Figure 9):

```
peakcheck/fit.py -- fit_amplitudes() (weighted NNLS fit of the amplitudes)
```

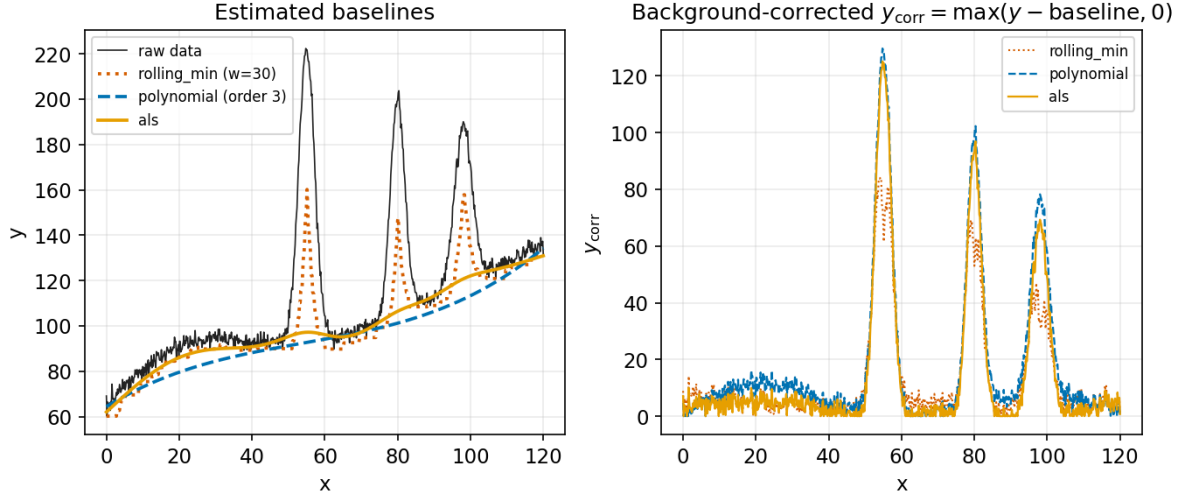


Figure 6. The three baseline estimators on the same curved, noisy background with three peaks. Left: the estimated baselines — the rolling minimum follows the lower envelope in discrete steps, while the polynomial (ModPoly) and the asymmetric-least-squares (ALS) curves are smooth. Right: the resulting corrected signal $y_{\text{corr}} = \max(y - \text{baseline}, 0)$; ALS and the polynomial flatten the inter-peak background more cleanly, whereas the rolling minimum leaves a little residual structure. All three preserve the peak areas.

```
def fit_amplitudes(x, y, yerr, centers, cfg, fit_mask=None):
    if len(centers) == 0:
        return np.array([])
    sigma, gamma = cfg.sigma(), cfg.gamma()
    y_corr = subtract_background(y, cfg.bg_window)
    basis = np.column_stack(
        [profile_shape(x, c, sigma, gamma, cfg.profile) for c in centers])
    if fit_mask is None:
        fit_mask = np.ones(len(x), dtype=bool)
    W = 1.0 / yerr
    A = basis[fit_mask] * W[fit_mask, np.newaxis]
    b = (y_corr * W)[fit_mask]
    amplitudes, _ = nnls(A, b)
    return amplitudes
```

$$\min_{A \geq 0} \|W(BA - y_{\text{corr}})\|_2^2, \quad B_{ik} = P_k(x_i), \quad W = \text{diag}(1/\sigma_i).$$

- The profile matrix **basis** holds one column $P_k(x_i)$ per peak (**profile_shape** selects Voigt/Gaussian/Lorentzian).
- $W = 1/\text{yerr}$ weights each point by the inverse uncertainty; **A** and **b** are the weighted design matrix and target vector.
- **fit_mask** restricts the fit to the window around the search range (plus a tail margin, **make_fit_mask()**) so distant structure cannot bias the amplitudes.
- `scipy.optimize.nnls` [12] solves the problem with the active-set algorithm of Lawson & Hanson [4]; non-negativity is physical (no negative intensities).
- Because the profiles are area-normalised, A_k is the integrated intensity of the peak.

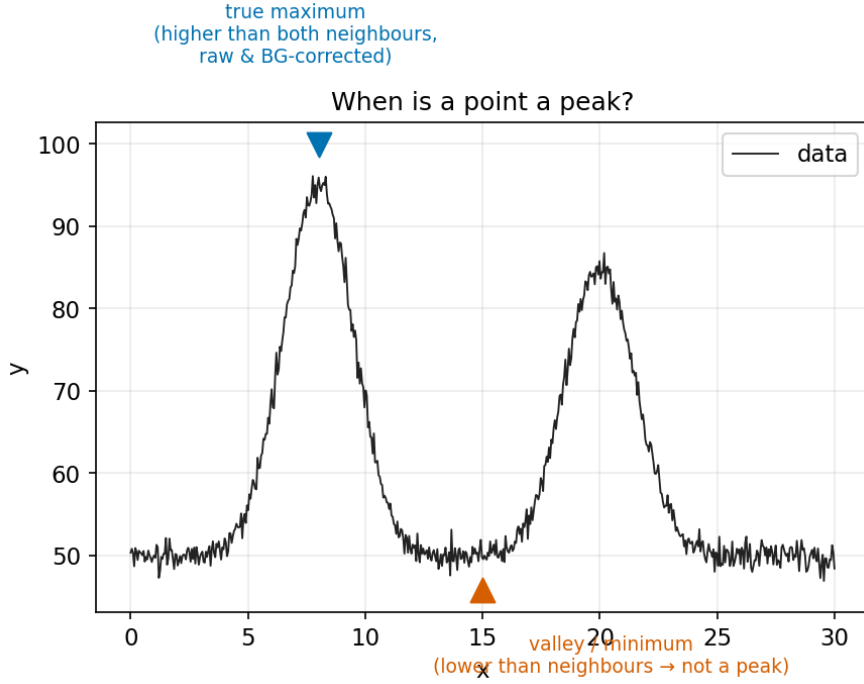


Figure 7. True maximum (green) versus a valley (red).

7.1 Fit window and model assembly

The fit runs not on the whole spectrum but on a window around the search range. `make_fit_mask()` widens the range by $3(f_G + f_L)$ so the profile wings are captured but distant structure cannot bias the amplitudes:

peakcheck/fit.py -- make_fit_mask() (fit window with a tail margin)

```
def make_fit_mask(x, cfg):
    margin = 3.0 * (cfg.wg + cfg.wl)
    lo = (cfg.x_min - margin) if cfg.x_min is not None else x[0]
    hi = (cfg.x_max + margin) if cfg.x_max is not None else x[-1]
    return (x >= lo) & (x <= hi)
```

From the fitted amplitudes, `compute_model()` reassembles the sum and the individual components — the same routine produces the curves for the plot and the Excel sheet:

peakcheck/fit.py -- compute_model() (sum model from amplitudes)

```
def compute_model(x, amplitudes, centers, sigma, gamma, profile):
    components = [amp * profile_shape(x, c, sigma, gamma, profile)
                  for amp, c in zip(amplitudes, centers)]
    total = np.sum(components, axis=0) if components else np.zeros_like(x)
    return total, components
```

7.2 The active-set method (NNLS)

The NNLS solver of Lawson & Hanson [4] maintains a set of “active” peaks (amplitude > 0) and a set pinned at the boundary ($A_k = 0$). It iteratively tests whether admitting a pinned peak lowers the weighted sum of squared residuals; otherwise the peak stays at zero. The result is the unique minimum under the constraint $\mathbf{A} \geq 0$. For PeakCheck this means a peak with no support in the data gets exactly $A_k = 0$ rather than a negative or noisy small value — physically correct, since negative intensities do not exist. This very mechanism makes the fit robust against over-specified

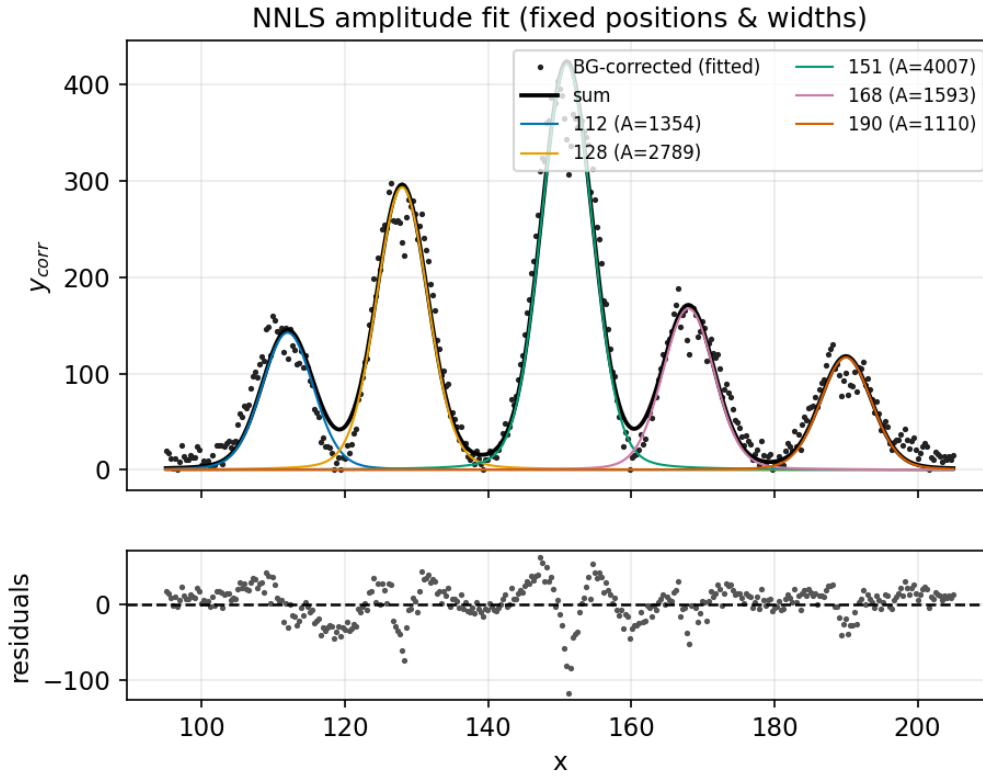


Figure 8. NNLS fit: sum (red) and individual components over the corrected data, with residuals below.

peaks.

8 Goodness of fit and standard errors

`fit_statistics()` assesses the fit and estimates the amplitude errors. The core block:

`peakcheck/fit.py -- fit_statistics()` (core block: χ^2 , covariance, error scaling)

```
chi2      = float(np.sum((resid * w) ** 2))
n_active  = int(np.sum(amplitudes > 0))
n_points  = int(len(yf))
dof        = max(n_points - n_active, 1)
chi2_red   = chi2 / dof

ss_res    = float(np.sum(resid ** 2))
ss_tot    = float(np.sum((yf - np.mean(yf)) ** 2))
r2        = 1.0 - ss_res / ss_tot if ss_tot > 0 else float("nan")

active    = np.where(amplitudes > 0)[0]
stderr_stat = np.full(n, np.nan)
if len(active) > 0:
    Ba = Bf[:, active] * w[:, np.newaxis]      # weighted design matrix
    try:
        cov = np.linalg.inv(Ba.T @ Ba)          # (B^T W^2 B)^-1
        stderr_stat[active] = np.sqrt(np.clip(np.diag(cov), 0.0, None))
    except np.linalg.LinAlgError:
        pass

if eff_mode == "scaled":
    scale = np.sqrt(chi2_red) if np.isfinite(chi2_red) and chi2_red > 0 else 1.0
else:
    scale = 1.0
```

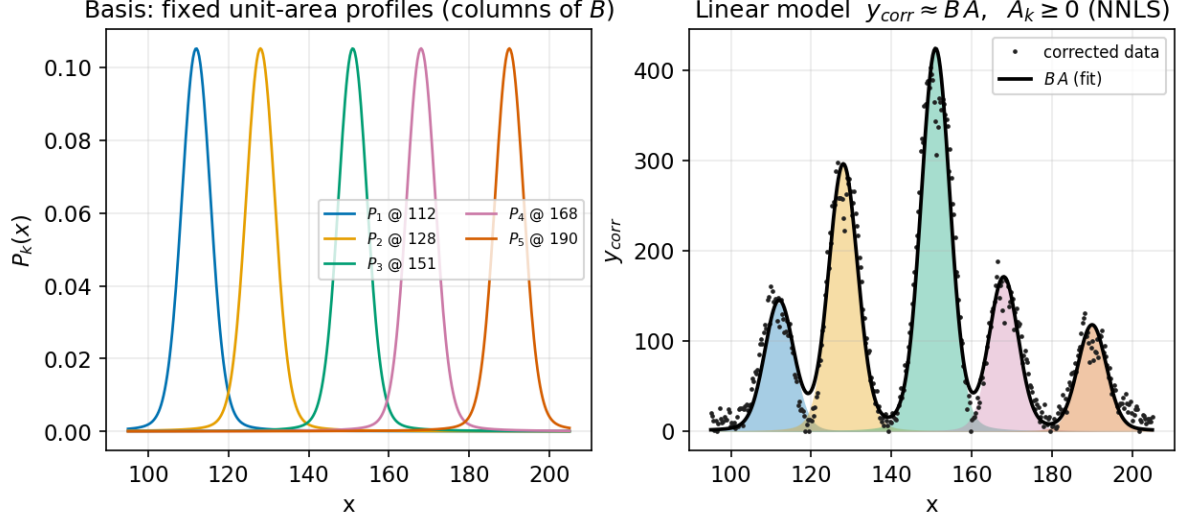


Figure 9. The amplitude fit is linear in the unknowns: the design matrix B has the fixed unit-area profiles as its columns (left), and the model is their non-negative combination BA (right). Only the heights A_k are free, which is what makes the problem a single convex NNLS solve.

```
stderr = stderr_stat * scale
```

$$\chi^2 = \sum_i \left(\frac{r_i}{\sigma_i} \right)^2, \quad \chi_\nu^2 = \frac{\chi^2}{N - N_{\text{active}} - N_{\text{extra}}}, \quad R^2 = 1 - \frac{\sum_i r_i^2}{\sum_i (y_i - \bar{y})^2}, \quad \text{Cov} = (B_{\text{act}}^\top W^2 B_{\text{act}})^{-1}.$$

Here N_{extra} is the number of additional free parameters beyond the amplitudes — the refined positions and/or per-peak widths (`n\_extra\_params` in `fit\_statistics`); it is 0 in the default fixed-position, fixed-width fit, so the formula reduces to the familiar $\chi^2/(N - N_{\text{active}})$. When refinement is enabled these extra parameters lower the degrees of freedom, so the reported χ_ν^2 accounts for them honestly (see Section 15).

- The covariance is formed over the active set ($A_k > 0$) only; `np.linalg.inv` gives $(B_{\text{act}}^\top W^2 B_{\text{act}})^{-1}$, whose square-rooted diagonal are the statistical 1σ errors [2, 6].
- `np.clip(..., 0, None)` guards against slightly negative diagonal values from rounding.
- For peaks pinned at the boundary ($A_k = 0$) the error stays `NaN` — the linear covariance does not apply there.

Two error modes — and why the choice is physical.

Mode	Reported error	When correct
statistical	σ_{A_k} (unscaled)	when the error column is a genuine 1σ (e.g. photon counting / Poisson): the random measurement error is reported honestly and χ_ν^2 stays a separate model-adequacy diagnostic
scaled	$\sigma_{A_k} \cdot \sqrt{\chi_\nu^2}$	when the errors are only relative weights of unknown scale (the convention of <code>curve_fit</code> , <code>absolute_sigma=False</code> [9])

Important: with genuine measurement errors, $\chi_\nu^2 > 1$ indicates *systematic* model error (one fixed width for all peaks, an approximate background, missing peaks) — not larger noise. Folding that systematic part into the random error bar via $\sqrt{\chi_\nu^2}$ would disguise a model problem as a measurement uncertainty. When no genuine errors exist (`had_errors=False`), `run_analysis()` switches to **scaled** automatically via `force_scaled`, since the absolute noise scale is then unknown.

9 Presence check in the components

For each reference peak, `check_peaks_in_component()` tests whether the component has a peak within $\pm \text{tolerance}$. The search has three stages:

1. **Strict maximum.** A local maximum in both the corrected and the raw data inside the (EDGE_PAD-extended) tolerance window; among several candidates the one with the highest corrected intensity wins.
2. **Shoulder fallback.** If stage 1 finds nothing, inflection points of dy/dx (via `np.gradient`) are tested that reach at least `SHOULDER_MIN_FRAC` = 50 % of the window maximum and are not a valley.
3. **Duplicate resolution.** If two references claim the same point, the closer one keeps it; the other searches for the next free maximum.

A candidate is accepted only if its signal-to-noise ratio reaches `snr_thresh` and $y_{\text{corr},i} > 0$. The SNR is evaluated by default on the background-corrected signal, $y_{\text{corr},i}/\sigma_i$ — the peak height *above* the background, which is the physically meaningful measure of presence (the background itself is not a peak). Setting `presence_baseline_corrected = false` reverts to the raw-signal ratio y_i/σ_i . There is no re-fitting here — only presence is determined.

9.1 The three stages in code

Stage 1 — strict maximum. Within the (edge-padded) tolerance window, local maxima of the corrected signal are located and accepted only if the point is also a maximum of the raw data (this prevents a point merely lifted by the background subtraction from passing as a peak). Among several candidates the one with the highest corrected intensity wins:

peakcheck/presence.py -- check_peaks_in_component(), stage 1

```
# Step 1: strict local maximum in the (edge-padded) window
ext_peaks, _ = find_peaks(y_ext_corr)          # maxima of corrected signal
valid_ext = [ep for ep in ext_peaks
              if 0 < ep < len(y_ext_raw) - 1
              and y_ext_raw[ep] >= y_ext_raw[ep - 1]  # also a raw maximum
              and y_ext_raw[ep] >= y_ext_raw[ep + 1]
              and abs(x_sub[ext_idx_ar[ep]] - center) <= tolerance]
if len(valid_ext) > 0:
    best_ep = valid_ext[int(np.argmax(y_ext_corr[valid_ext]))] # highest wins
    imax    = int(ext_idx_ar[best_ep])
```

Stage 2 — shoulder fallback. If stage 1 finds nothing, inflection points are sought via the derivative dy/dx (with `np.gradient`). A shoulder counts only if it reaches at least `SHOULDER_MIN_FRAC` = 50 % of the window maximum and is not a valley in the raw data:

peakcheck/presence.py -- check_peaks_in_component(), stage 2

```
# Step 2: shoulder fallback (only if Step 1 found nothing)
if imax is None and len(window_idx) >= 3:
    dy = np.gradient(y_win_corr.astype(float))
    rising_cands, _ = find_peaks(dy)          # inflection points of dy/dx
    falling_cands, _ = find_peaks(-dy)
    all_cands = np.concatenate([rising_cands, falling_cands])

    win_max_corr = np.max(y_win_corr)
    valid_shoulders = []
    for c in all_cands:
        if c == 0 or c == len(y_win_raw) - 1:
            continue
        if y_win_corr[c] < SHOULDER_MIN_FRAC * win_max_corr:  # >= 50 %
```

```

        continue
    if y_win_raw[c] < y_win_raw[c - 1] and y_win_raw[c] < y_win_raw[c + 1]:
        continue
    valid_shoulders.append(c)

    if len(valid_shoulders) > 0:
        shoulders_arr = np.array(valid_shoulders)
        best_local = shoulders_arr[np.argmax(y_win_corr[shoulders_arr])]
        imax = int(window_idx[best_local])

```

A located point is then accepted if it clears the SNR threshold — on the signal justified above (y_{snr} is corrected or raw):

peakcheck/presence.py -- check_peaks_in_component(), acceptance

```

snr = y_snr[imax] / yerr_sub[imax] # y_snr = corrected or raw
found = (snr >= snr_thresh) and (y_corr_sub[imax] > 0)

```

Stage 3 — duplicate resolution. If two closely spaced references claim the same data point, the nearer reference keeps it; the other searches for the next free maximum in its own window (this prevents a single peak from being counted twice):

peakcheck/presence.py -- check_peaks_in_component(), stage 3

```

# Step 3: duplicate resolution -- two references claim the same sample
j = taken[imax_i]
dist_i = abs(x_sub[imax_i] - peak_centers[i])
dist_j = abs(x_sub[imax_i] - peak_centers[j])
winner, loser = (i, j) if dist_i <= dist_j else (j, i) # nearer reference keeps it
taken[imax_i] = winner
# the loser searches the next free local maximum in its own window:
alt_peaks, _ = find_peaks(y_corr_sub[ext_l])
alt_valid = [ep for ep in alt_peaks
              if 0 < ep < len(ext_l) - 1
              and y_sub[ext_l[ep]] >= y_sub[ext_l[ep] - 1]
              and y_sub[ext_l[ep]] >= y_sub[ext_l[ep] + 1]
              and abs(x_sub[ext_l[ep]] - center_l) <= tolerance
              and int(ext_l[ep]) not in taken
              and int(ext_l[ep]) != imax_i]

```

This three-stage logic is deliberately conservative: it accepts only genuine maxima or clear shoulders and resolves ambiguities geometrically (by proximity to the reference position) rather than arbitrarily.

10 Graphical interface

PeakCheckGUI embeds a Matplotlib [3] plot in a Tkinter window. The automatic search provides the starting point; the user keeps control by mouse. A fourth control row exposes the baseline method (Section 5) and the optional position refinement (Section 14), so both are reachable from the GUI as well as the TOML. The central click handler:

peakcheck/gui.py -- PeakCheckGUI._on_click() (add/remove peaks by mouse)

```

def _on_click(self, event):
    if self.x is None or event.inaxes != self.ax or event.xdata is None:
        return
    idx = int(np.argmin(np.abs(self.x - event.xdata)))
    if event.button == 1: # add
        if idx not in self.peaks:
            self.peaks.append(idx)

```

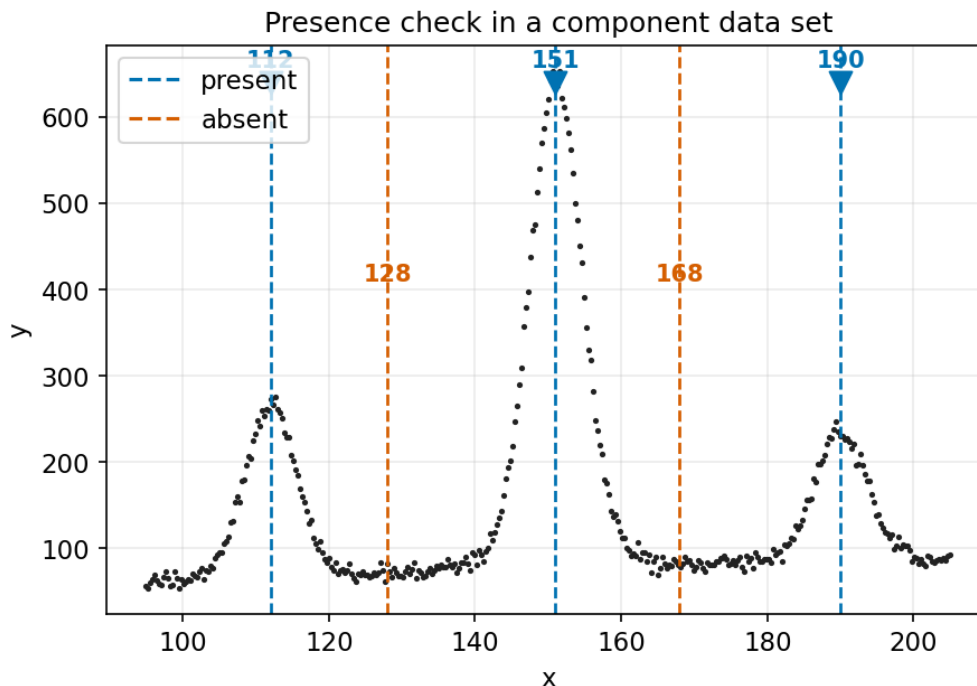


Figure 10. Presence check: present peaks green, absent red; dashed lines mark the reference positions.

```

        self.peaks.sort()
        self._redraw()
    elif event.button == 3:          # remove nearest
        if self.peaks:
            clicked_x = self.x[idx]
            peak_xs = self.x[self.peaks]
            nearest = self.peaks[int(np.argmin(np.abs(peak_xs - clicked_x)))]
            if abs(self.x[nearest] - clicked_x) < (self.x[-1] - self.x[0]) * 0.04:
                self.peaks.remove(nearest)
                self._redraw()

```

- `np.argmin(np.abs(self.x - event.xdata))` finds the data point nearest the click.
- Left click (`button == 1`) adds that index as a peak (if not already present).
- Right click (`button == 3`) removes the nearest existing peak, provided it is close enough (within 4% of the *x* span).
- `_redraw()` updates markers and labels immediately.

11 Outputs and reproducibility

`save_excel()` writes up to six sheets; `save_csv()` writes the same data as a single plain 7-bit ASCII file `*_results.csv` (enforced by `_ascii()`) with five labelled sections. Both carry the full parameter block from `gather_parameters()` as a header.

Every run is thus fully documented and reconstructable from its config (or the CSV header) alone.

11.1 Presence matrix with colour coding

The **Presence** sheet writes a 1 (green) or 0 (red) per peak and component and colours the cell accordingly, so the key result is readable at a glance:

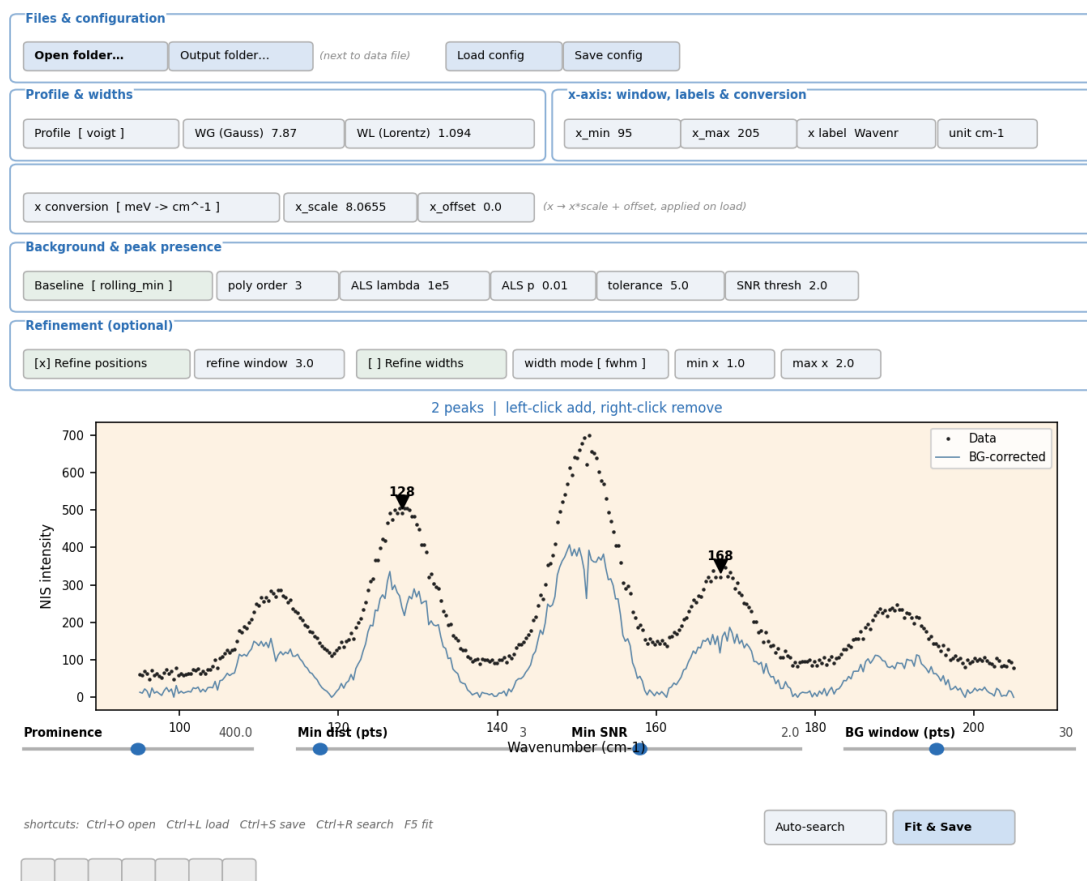


Figure 11. The PeakCheck window: control bars, plot with peak markers, sliders and toolbar.

peakcheck/output.py -- save_excel(), Presence sheet

```
FOUND_FILL    = PatternFill("solid", start_color="C6EFCE")    # green
NOTFOUND_FILL = PatternFill("solid", start_color="FFC7CE")    # red
# ... for each (peak, component) cell:
c = ws3.cell(row=i, column=j)
if fmask[idx]:
    c.value, c.fill, c.font = 1, FOUND_FILL, FOUND_FONT
else:
    c.value, c.fill, c.font = 0, NOTFOUND_FILL, NOTFOUND_FONT
```

11.2 Plain ASCII CSV with a parameter header

The CSV file is deliberately written as 7-bit ASCII (via `errors="replace"` and `_ascii()` for the column names) so it reads without encoding issues on any system and in any pipeline. The single `*_results.csv` holds five labelled sections — Intensities, Positions, Presence, Fit_Data and Components_Raw — separated by `## == SECTION: <name> ==` lines, and starts with the full parameter block as a commented header:

peakcheck/output.py -- save_csv(), ASCII enforcement & header

```
def _ascii(s):
    return str(s).encode("ascii", "replace").decode("ascii")    # force 7-bit ASCII

# one results file; component column labels are forced to ASCII:
comp_labels = [_ascii(os.path.splitext(os.path.basename(f))[0])
               for f, _, _, _ in component_results]
```

Sheet	Content
Intensities	peak, NNLS amplitude, standard error, measured intensity per component
Positions	reference and measured positions (tracks shifts)
Presence	binary matrix 1 (green) / 0 (red) — the key result
Fit_Data	raw data, errors, corrected, sum and individual components
Components_Raw	each component's raw (x, y, error) as side-by-side blocks (only when components are analysed)
Parameters	full run metadata (path, profile, widths, window, error mode, χ^2/R^2 , versions)

```
# the file opens in ASCII mode and starts with the parameter block:
with open(out_path, "w", newline="", encoding="ascii", errors="replace") as fh:
    for ln in _csv_header_lines(params):
        fh.write(ln + "\n")
```

11.3 What the parameter block contains

`gather_parameters()` collects all run metadata into an ordered table — input path, axis transform, error provenance, profile and widths, windows, χ^2/R^2 and the versions of numpy/scipy/Python. When width refinement is enabled it also records the mode, the width bounds and the resulting per-peak FWHM and scale factors. Every run is thus fully documented:

peakcheck/output.py -- gather_parameters() (excerpt)

```
p["Reference file"] = os.path.abspath(cfg.reference_file)
p["x transform"] = f"x*{cfg.x_scale} + {cfg.x_offset}"
p["Errors from file"] = "yes" if had_errors else "no (estimated from data)"
p["Profile"] = str(cfg.profile).lower()
p[f"Effective FWHM{unit}"] = round(cfg.effective_fwhm(), 4)
p["chi2_reduced"] = round(stats["chi2_red"], 4)
p["R2"] = round(stats["r2"], 6)
p["numpy"] = np.__version__
p["scipy"] = scipy.__version__
p["Python"] = sys.version.split()[0]
```

This block goes into both the Excel **Parameters** sheet and the header of every CSV file. Together with the saved TOML configuration, a run is therefore fully reproducible.

12 Worked example

The bundled example (`example_data/`) shows the complete workflow on synthetic NIS data [8, 10, 11]. The reference `sample.nis` contains five peaks; the x values are stored in meV. The configuration `nis_example.toml` selects `x_conversion = "meV -> cm-1"`, so the whole analysis runs in cm^{-1} .

1. Read & transform. The meV axis (about 11.8–25.4 meV) is mapped by the factor 8.065544 into the range 95–205 cm^{-1} .

2. Fit. With fixed positions [112, 128, 151, 168, 190] cm^{-1} and the instrumental widths ($w_g = 7.87$, $w_l = 1.09$), the weighted NNLS fit returns the integrated intensities:

Peak (cm^{-1})	112	128	151	168	190
Amplitude A_k	1969	4111	5675	2408	1507

The fit quality is $\chi_\nu^2 = 5.5$ at $R^2 = 0.945$. The elevated χ_ν^2 is intentional and instructive here: the data carry genuine Poisson errors, so any model simplification (one fixed width for all peaks) shows up as a systematic contribution to the reduced chi-square — exactly the situation of Section 8 in which `statistical` is the correct error mode.

3. Presence. In the component `sample_A.nis` the peaks at 128 and 168 cm^{-1} are absent by construction. The check on the background-corrected signal separates them clearly (threshold `snr_thresh = 5`):

Peak (cm^{-1})	y_{corr}	SNR_{corr}	Result
112	227.6	13.8	present (1)
128	17.4	4.4	absent (0)
151	490.5	18.9	present (1)
168	14.3	3.8	absent (0)
190	143.8	11.4	present (1)

The background-corrected signal separates by more than an order of magnitude (present ~ 140 – 490 vs absent ~ 15), whereas the raw signal would be more similar at the same positions because of the background — the choice of signal justified in Section 9 makes the decision robust. The presence matrix in the Excel sheet accordingly reads $A = [1, 0, 1, 0, 1]$.

13 Numerical validation and exactness

Several of the claims above can be checked directly against reference implementations and against the bundled example.

The profile is the true Voigt. Evaluated on a wide grid, PeakCheck’s `voigt()` agrees with `scipy.special.voigt_profile` to floating-point precision: the maximum absolute deviation is about 3×10^{-17} and the maximum relative deviation about 1×10^{-14} . The two are therefore identical up to rounding — confirming that the profile is the exact convolution and not a pseudo-Voigt, whose deviation from the true shape is percent-level (Figures 3 and 4).

Area normalisation. Each profile integrates analytically to one. A finite-range numerical integral approaches one as the range grows; because the Lorentzian wings decay only as $1/x^2$, a modest window leaves a few percent of the area in the tails, which is why the fit window is widened by $3(f_G + f_L)$ (Section 7.1) so that the dominant part of every profile is inside it.

Uniqueness of the amplitude fit. The weighted NNLS objective $\|W(B\mathbf{A} - \mathbf{y}_{\text{corr}})\|_2^2$ is a convex quadratic (its Hessian $B^\top W^2 B$ is positive semidefinite) on the convex feasible set $\mathbf{A} \geq 0$, so the minimiser is unique whenever the active profile columns are linearly independent. The active-set method of Lawson & Hanson [4] reaches it in finitely many steps, and the solution satisfies the Karush–Kuhn–Tucker conditions: an active peak ($A_k > 0$) has a vanishing gradient component, a pinned peak ($A_k = 0$) a non-negative one. A peak that the data do not support therefore receives exactly $A_k = 0$ rather than a noisy small or negative value.

Determinism and reproducibility. The computational core contains no random component, so a given configuration — equivalently, the parameter block stored in the Excel `Parameters` sheet or the CSV header — reproduces the result on the same platform. The bundled example doubles as a regression check: with the shipped `nis_example.toml` the fit returns the amplitudes, χ_ν^2 , R^2 and presence matrix quoted in Section 12, and the test suite asserts the core numerics.

13.1 Monte-Carlo recovery and error calibration

The covariance estimate of Section 16 can be checked directly. Injecting the five amplitudes of the worked example into the noiseless model, adding independent Gaussian noise with the per-point σ_i , and re-running the NNLS fit $N = 600$ times yields the recovered amplitudes of Figure 12. The mean recovered amplitudes coincide with the injected values — the largest bias is 0.06σ , i.e. the estimator is unbiased to within Monte-Carlo error — and the empirical scatter (the standard deviation across the N trials) reproduces the analytic $\sigma = \sqrt{(B^\top W^2 B)_{kk}^{-1}}$ to within 6% for every peak. The linear fit therefore returns both the correct amplitudes and a quantitatively trustworthy error budget whenever the model and the noise estimate are correct.

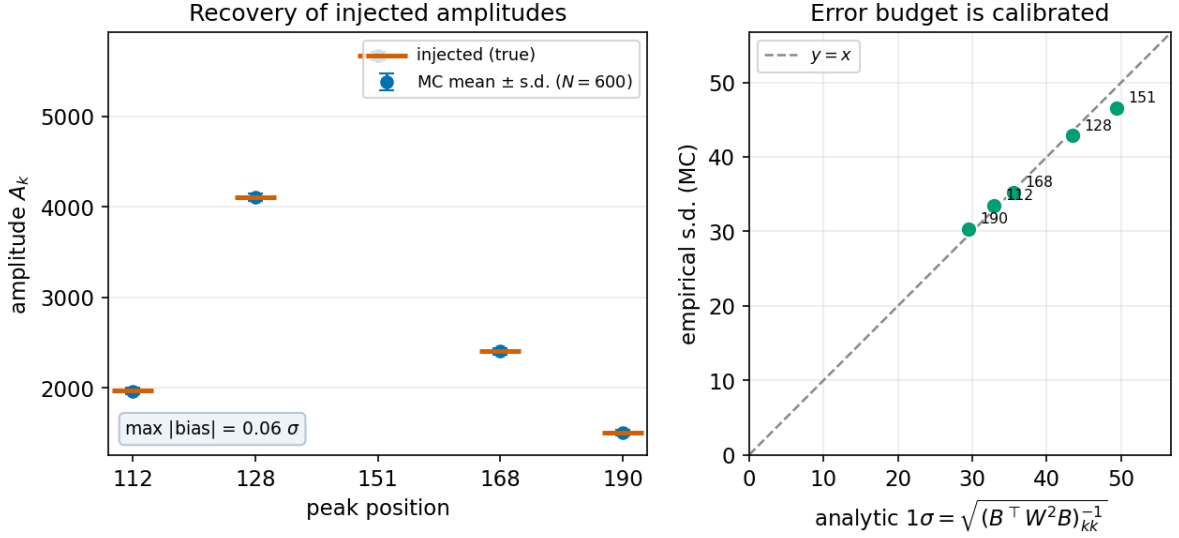


Figure 12. Monte-Carlo recovery over $N = 600$ noise realisations. Left: the mean recovered amplitudes (points, with $\pm$ one empirical standard deviation) sit on the injected values (ticks); the maximum bias is 0.06σ . Right: the empirical scatter versus the analytic 1σ from the covariance lies on the $y = x$ line, confirming that the reported errors are calibrated.

14 Assumptions, limitations and parameter selection

PeakCheck is intentionally a focused tool; knowing what it does *not* do is part of using it correctly.

- **Fixed positions and widths.** The fit is linear — and hence fast, unique and robust — precisely because the peak positions and the two widths are held fixed. PeakCheck quantifies intensities at known or expected positions; it does not refine positions or per-peak widths *by default*. An optional refinement (`refine = true`) relaxes this for the positions only: holding the widths fixed, it refines the centres by a bounded variable-projection fit (Section 15), each centre constrained to a small window so it cannot migrate onto a neighbour. If the instrumental resolution is misestimated or peaks overlap strongly, the fixed-width model absorbs the mismatch into the amplitudes and into an elevated χ^2_ν (Section 8).
- **One global noise scale.** Without an error column, a single σ is estimated from the high-frequency differences (Section 3). This suits roughly homoscedastic noise; for strongly count-dependent (Poisson) data spanning orders of magnitude, supply a genuine error column so the weights $1/\sigma_i$ and the reported errors are correct.
- **Rolling-minimum background.** The baseline model is parameter-light and robust to peak shape, but it assumes the background varies more slowly than a peak. A window narrower

than a peak eats into it; a window wider than the background curvature leaves a residual slope (Figure 13).

- **Presence is a detection, not a fit.** The presence check decides only whether a peak is present in a component within the tolerance window; it does not re-fit the component. Its multi-stage logic is deliberately conservative (true maxima, then clear shoulders, with geometric duplicate resolution).

The handful of parameters that matter most, and how to choose them:

Parameter	Effect and how to choose
<code>bg_window</code>	baseline window: wider than a peak FWHM (in points), narrower than the background curvature; 0 disables it
<code>prominence_init</code>	search sensitivity: raise to reject noise, lower to catch weak peaks; tune live in the GUI
<code>distance_init</code>	minimum peak separation in points: $\approx$ the smallest spacing you expect to resolve
<code>min_snr_init</code> / <code>snr_thresh</code>	SNR gates for search / presence: 2–3 for clean data, 4–6 for noisy spectra
<code>tolerance</code>	presence window: larger than expected centroid shifts, smaller than the peak spacing
<code>error_mode</code>	statistical for genuine 1σ errors; scaled for relative weights of unknown scale (Section 8)

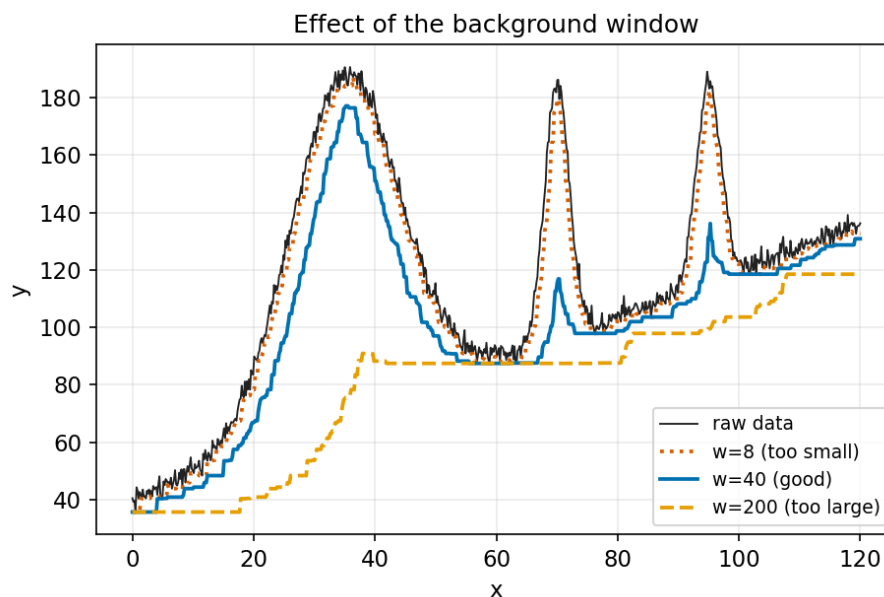


Figure 13. Choosing the background window. A window much narrower than a peak follows the peak flanks and is subtracted away (too small); a window much wider than the baseline curvature cannot track the slope and leaves a residual ramp (too large). A window a few times the peak width tracks the baseline while leaving the peaks intact.

15 Relation to nonlinear least-squares fitting

PeakCheck’s central design choice — fixing the peak positions and the two widths and fitting only the amplitudes by NNLS — is what separates it from general nonlinear peak fitting (Levenberg–Marquardt as in `scipy.optimize.curve_fit` [9], or `lmfit`), which refines positions and widths as well.

- **Convexity and uniqueness.** With positions and widths fixed the residual is linear in $\mathbf{A}$, the objective is convex, and NNLS returns the unique global optimum — no starting values, no local minima, no convergence failures. A nonlinear fit over positions/widths is non-convex (Figure 14): it needs good initial guesses and can settle in a local minimum.
- **Non-negativity.** NNLS enforces $A_k \geq 0$ exactly, so an absent mode gets $A_k = 0$. A nonlinear solver returns whatever minimises the residual, including unphysical negative areas, unless explicitly bounded.
- **Speed.** One NNLS solve replaces an iterative sequence of Jacobian evaluations (Section 17).
- **What is given up.** PeakCheck does *not* refine positions or widths; it assumes the instrumental resolution (the two widths) is known and the positions are given or picked in the GUI. When those are themselves unknown or variable, a full nonlinear fit is the right tool; an elevated χ^2_ν (Section 8) is the signal that the fixed-shape assumption is being strained.

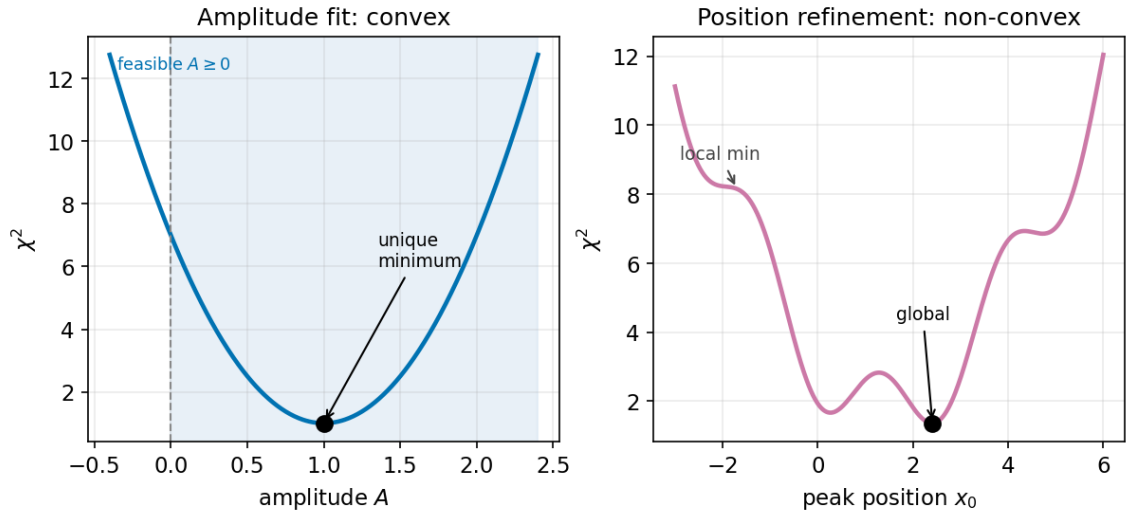


Figure 14. Why fixing the shape helps. Left: with positions and widths fixed the objective is convex in the amplitude, so there is a single minimum on the feasible region $A \geq 0$. Right: refining a peak position makes the objective non-convex, with local minima a solver can fall into.

	PeakCheck (fixed-shape NNLS)	general nonlinear fit
free parameters	amplitudes only	positions, widths, amplitudes
objective	convex — unique optimum	non-convex — local minima
initial guess	none needed	required, sensitive
non-negativity	guaranteed	only if bounded
positions/widths	fixed (assumed known)	refined
typical use	intensity quantification at known modes	full line-shape characterisation

Table 1. When the modes and the resolution are known, the fixed-shape NNLS fit is more robust and reproducible; when they are not, a nonlinear fit is appropriate.

A middle ground. The optional position refinement (**refine**, Section 14) is a deliberately small step in the nonlinear direction: the amplitudes are still obtained by NNLS through variable projection, only the centres are refined, and only within a bounded window. It recovers slightly-offset positions without surrendering the non-negativity and robustness of the linear fit, but it does not replace a full line-shape analysis when the widths themselves are unknown. Figure 15 illustrates the effect.

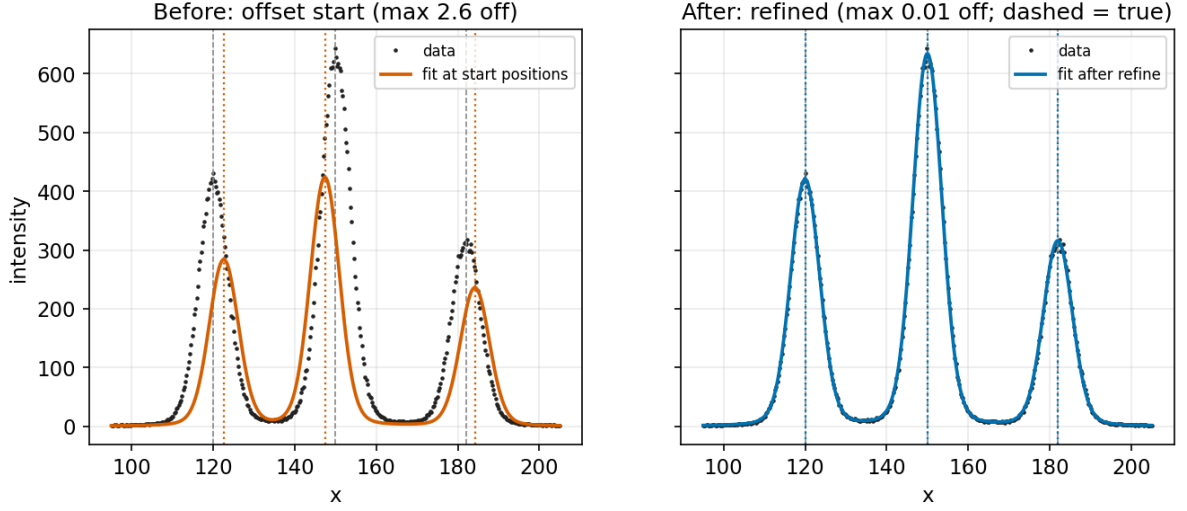


Figure 15. The optional position refinement. Left: with deliberately offset start centres (dotted) the fixed-position fit is visibly misaligned with the true centres (dashed grey). Right: with `refine = true` the bounded variable-projection step recovers the centres to a small fraction of a sample spacing, while the amplitudes remain non-negative throughout.

Per-peak widths. The same variable-projection idea extends to the widths. With `refine\_widths = true` one width *scale* per peak is introduced and refined by the same bounded least-squares solve, the amplitudes again obtained by NNLS for every trial set of widths. `width\_mode` selects whether the whole Voigt is scaled ("`fwhm`": both σ and γ , so the effective FWHM scales with the factor — the Olivero–Longbothum width is homogeneous of degree one in (f_G, f_L)) or only the Gaussian part ("`sigma`": inhomogeneous broadening, the Lorentzian lifetime width fixed). Each scale is constrained to `[width_min_factor, width_max_factor]` times the instrumental width, with the lower bound defaulting to the instrument itself so a band can never be reported narrower than the resolution. Because every refined width is an extra free parameter, the degrees of freedom are reduced accordingly (`n\_extra\_params` in `fit\_statistics`), so χ^2_ν is computed honestly; even so, freeing widths *always* lowers χ^2_ν , and for overlapping or weak peaks width and amplitude (or a neighbour’s intensity) trade off, which is exactly why the bounds matter. Figure 16 shows the recovery on a genuinely broadened peak.

16 Error budget: a worked covariance example

The per-amplitude uncertainties are the square roots of the diagonal of the linear covariance matrix. For the active peaks ($A_k > 0$) with weighted design matrix B_{act} and $W = \text{diag}(1/\sigma_i)$,

$$C = (B_{\text{act}}^\top W^2 B_{\text{act}})^{-1}, \quad \sigma_{A,k} = \sqrt{C_{kk}}, \quad \rho_{ij} = \frac{C_{ij}}{\sqrt{C_{ii} C_{jj}}},$$

and in `scaled` mode each $\sigma_{A,k}$ is multiplied by $\sqrt{\chi^2_\nu}$ (Section 8). The off-diagonal correlations ρ_{ij} quantify how much two amplitudes trade off against each other: strongly overlapping peaks are anticorrelated (raising one can be partly absorbed by lowering its neighbour), whereas well-separated peaks are nearly independent.

For the bundled example (Section 12, `statistical` mode) the fitted amplitudes and their 1σ errors are 1969 ± 33 , 4111 ± 45 , 5675 ± 52 , 2408 ± 36 and 1507 ± 27 . Figure 17 shows the amplitudes with these error bars and the corresponding correlation matrix. Because the peaks here are well separated, the off-diagonal correlations are small ($|\rho| \lesssim 0.05$) and the amplitudes are effectively independent — the diagonal errors can be read off directly. For closely spaced modes the same

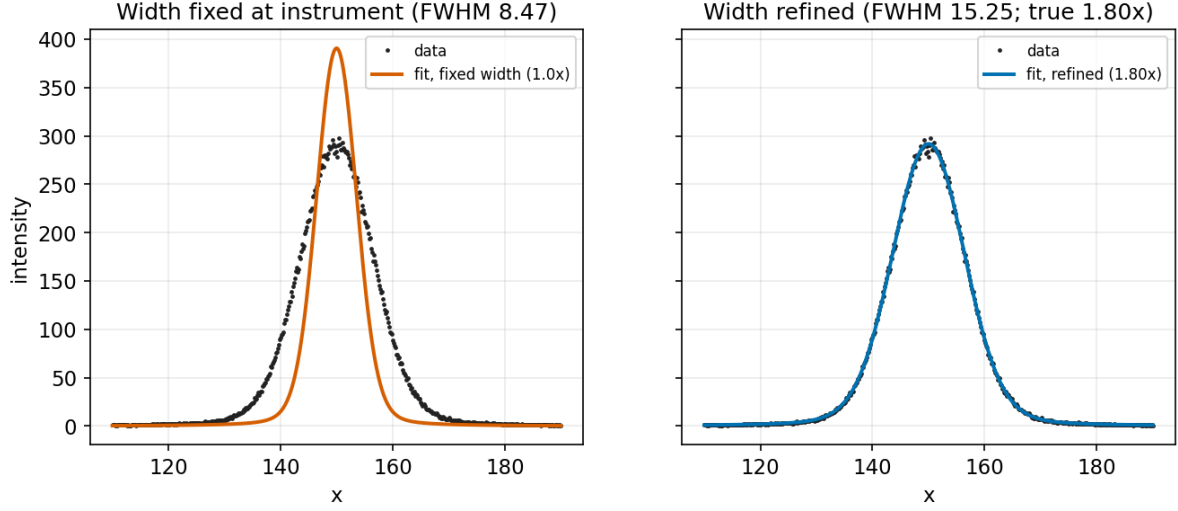


Figure 16. Optional per-peak width refinement. Left: with the width fixed at the instrumental resolution the fit undershoots a peak that is genuinely broader. Right: with `refine\_widths = true` the bounded variable-projection step recovers the per-peak width (here close to the true broadening factor) while the amplitude stays non-negative.

matrix would show pronounced anticorrelations, a warning that the individual amplitudes are less well determined than their diagonal errors alone suggest. Statistical mode corresponds to `curve_fit(..., absolute_sigma=True)` and scaled mode to `absolute_sigma=False`.

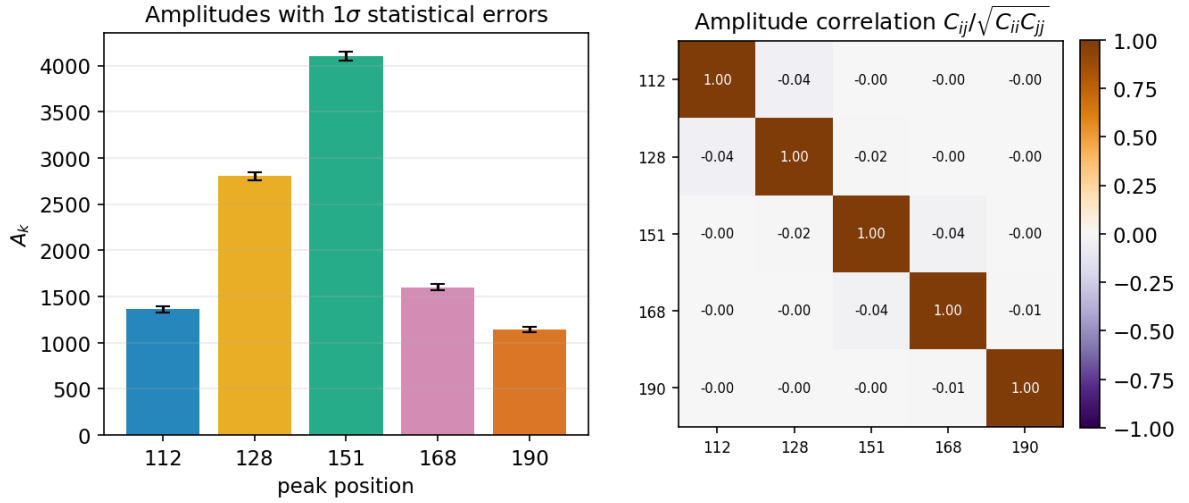


Figure 17. Left: fitted amplitudes with 1σ statistical errors from $\sqrt{C_{kk}}$. Right: the amplitude correlation matrix $\rho_{ij} = C_{ij} / \sqrt{C_{ii} C_{jj}}$ for a representative five-peak spectrum; the small off-diagonal values reflect the well-separated peaks.

17 Computational complexity and performance

The pipeline is linear in the spectrum length and modest in the number of peaks. Building the design matrix B costs $O(NK)$ profile evaluations (N points, K peaks; each a Faddeeva call). Forming the weighted normal equations is $O(NK^2)$, and the active-set NNLS solve then works on the small $K \times K$ system, with K typically a few to a few tens. The peak search (`find_peaks`) and the rolling-minimum background are $O(N)$. Memory is $O(NK)$ for B , negligible at spectroscopic sizes.

Figure 18 confirms this with measured timings: the end-to-end fit (build B + solve) grows linearly with N from a few hundred to 5×10^4 points, and even at the largest size it takes only tens of milliseconds; the NNLS solve itself stays well below a millisecond, since it sees only the K columns. Re-fitting on every GUI slider change is therefore effectively instantaneous, and the dominant cost in practice is file I/O and plotting rather than the numerics.

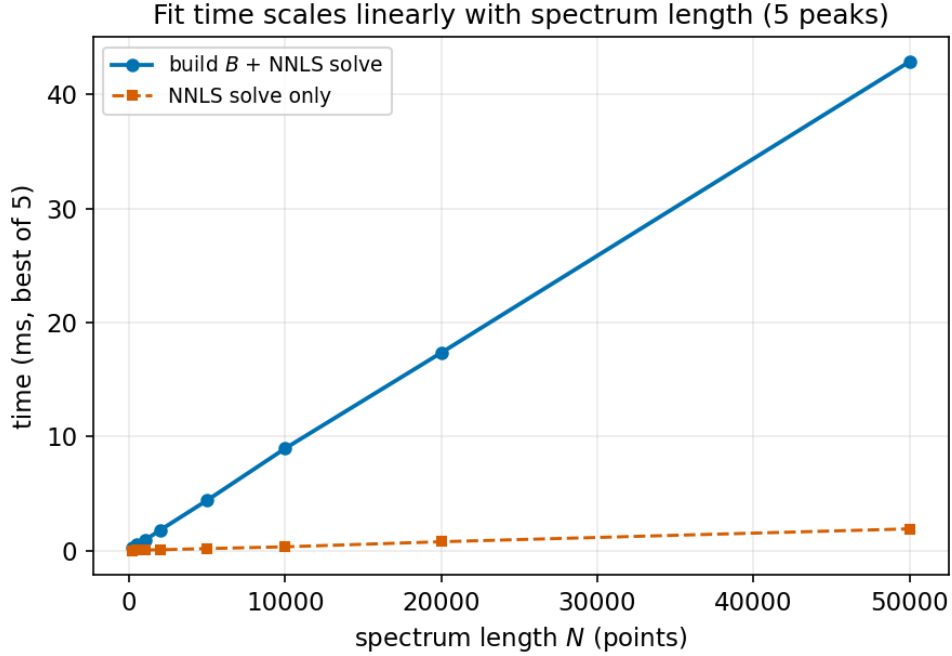


Figure 18. Measured fit time versus spectrum length (five peaks, best of five runs). Building B and solving the NNLS together scale linearly with N ; the NNLS solve itself is sub-millisecond because it only sees the K profile columns.

18 Parameter reference

All `Config` parameters with their meaning (in TOML-section order):

Since TOML has no `null`, empty fields are turned into `None` on load by `_coerce_nullable()`:

`peakcheck/config.py -- _coerce_nullable() (empty strings to None)`

```
def _coerce_nullable(cfg: Config) -> None:
    for key in ("x_min", "x_max"):
        v = getattr(cfg, key)
        if v == "" or v is None:
            setattr(cfg, key, None)
        else:
            setattr(cfg, key, float(v))
```

19 References

- [1] Milton Abramowitz and Irene A. Stegun. *Handbook of Mathematical Functions*. Dover Publications, New York, 1972. Sec. 7.1 (Faddeeva function and the Voigt profile).
- [2] Philip R. Bevington and D. Keith Robinson. *Data Reduction and Error Analysis for the Physical Sciences*. McGraw-Hill, New York, 3 edition, 2003.

Parameter	Meaning
reference_file	data set in which the peaks are defined
component_glob	component search pattern; empty = <name>_*
error_column	"auto" / "none" / column index
fio_x_column, fio_y_column	FIO only: x/y column by name or 1-based index (default y "nisp")
x_conversion	named preset (e.g. "meV -> cm ⁻¹ ")
x_transform	"affine" or "reciprocal"
x_scale, x_offset	factor/offset or numerator of the transform
x_label, y_label, x_unit	axis labels and x-axis unit string (display only)
profile	"voigt" / "gauss" / "lorentz"
wg, wl	Gaussian and Lorentzian FWHM
x_min, x_max	search range (target unit); empty = full range
bg_window	background window size (points); 0 = off
baseline_method	"rolling_min" / "polynomial" / "als"
baseline_poly_order	order of the polynomial baseline
baseline_als_lambda, baseline_als_p	ALS smoothness and asymmetry
prominence_init, distance_init, min_snr_init	starting values of the automatic search
refine, refine_window	refine positions (bounded); max shift in x-units
refine_widths, width_mode	refine per-peak widths; "fwhm" or "sigma"
width_min_factor, width_max_factor	width bounds as factors of the instrumental width
tolerance	± window of the presence check (target unit)
snr_thresh	SNR threshold of the presence check
presence_baseline_corrected	SNR on corrected (true) or raw (false) signal
error_mode	"statistical" or "scaled"
output_dir	target folder for outputs; empty = next to the reference file
write_csv, write_excel	write the CSV / Excel outputs (both default true)
plot_dpi	resolution (dpi) of the saved plots
peaks	optional fixed peak positions (otherwise search/-mouse)

- [3] John D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007. doi: 10.1109/MCSE.2007.55.
- [4] Charles L. Lawson and Richard J. Hanson. *Solving Least Squares Problems*. Classics in Applied Mathematics. Society for Industrial and Applied Mathematics (SIAM), Philadelphia, 1995. doi: 10.1137/1.9781611971217. Ch. 23; originally published by Prentice-Hall, 1974.
- [5] J. J. Olivero and R. L. Longbothum. Empirical fits to the Voigt line width: A brief review. *Journal of Quantitative Spectroscopy and Radiative Transfer*, 17(2):233–236, 1977. doi: 10.1016/0022-4073(77)90161-3.
- [6] William H. Press, Saul A. Teukolsky, William T. Vetterling, and Brian P. Flannery. *Numerical Recipes: The Art of Scientific Computing*. Cambridge University Press, Cambridge, 3 edition, 2007. Ch. 15 (modeling of data; least-squares covariance).
- [7] Peter J. Rousseeuw and Christophe Croux. Alternatives to the median absolute deviation. *Journal of the American Statistical Association*, 88(424):1273–1283, 1993. doi: 10.1080/01621459.1993.10476408.
- [8] W. R. Scheidt, S. M. Durbin, and J. T. Sage. Nuclear resonance vibrational spectroscopy –

- NRVS. *Journal of Inorganic Biochemistry*, 99(1):60–71, 2005. doi: 10.1016/j.jinorgbio.2004.11.004.
- [9] SciPy developers. `scipy.optimize.curve_fit` — documentation (parameter `absolute_sigma`). https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.curve_fit.html, 2024. Accessed 2026.
- [10] W. Sturhahn. Nuclear resonant spectroscopy. *Journal of Physics: Condensed Matter*, 16(5): S497–S530, 2004. doi: 10.1088/0953-8984/16/5/009.
- [11] W. Sturhahn, T. S. Toellner, E. E. Alp, X. Zhang, M. Ando, Y. Yoda, S. Kikuta, M. Seto, C. W. Kimball, and B. Dabrowski. Phonon density of states measured by inelastic nuclear resonant scattering. *Physical Review Letters*, 74(19):3832–3835, 1995. doi: 10.1103/PhysRevLett.74.3832.
- [12] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3):261–272, 2020. doi: 10.1038/s41592-019-0686-2.

20 Notation

Symbol	Meaning
x, x_0	abscissa (working units) and peak centre
σ	Gaussian standard deviation
γ	Lorentzian half width at half maximum
$f_G = 2\sqrt{2\ln 2}\sigma$	Gaussian FWHM (<code>wg</code>)
$f_L = 2\gamma$	Lorentzian FWHM (<code>wl</code>)
f_V	total Voigt FWHM (Olivero–Longbothum)
G, L, V	Gaussian, Lorentzian, Voigt profiles (each of unit area)
$w(z) = e^{-z^2} \operatorname{erfc}(-iz)$	Faddeeva function (<code>scipy.special.wofz</code>)
$z = \frac{(x - x_0) + i\gamma}{\sigma\sqrt{2}}$	complex argument of w
P_k	area-normalised profile of peak k
$B, B_{ik} = P_k(x_i)$	profile (design) matrix
A_k	fitted amplitude = integrated intensity of peak k
$W = \operatorname{diag}(1/\sigma_i)$	diagonal weight matrix
σ_i	per-point 1σ uncertainty
$\mathbf{y}, \mathbf{y}_{\text{corr}}$	raw and background-corrected intensities
r_i	residual (corrected data minus model)
N, N_{active}	points in the fit window; peaks with $A_k > 0$
N_{extra}	extra free parameters (refined positions/widths; 0 by default)
χ^2, χ_ν^2	chi-square and reduced $\chi^2 = \chi^2/(N - N_{\text{active}} - N_{\text{extra}})$
R^2	coefficient of determination
η	pseudo-Voigt mixing fraction (shown for contrast; not used by PeakCheck)